AI ASSISTED CODING

ASSIGNMENT-11.5

NAME: M.AKHIL REDDY

HALL TICKET:2303A52315

BATCH-45

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

PROMPT:" Generate a Python Stack class with methods push, pop, peek, and is_empty. Include docstrings explaining each method clearly"

CODE:

```python
#Generate a Python Stack class with methods push, pop, peek, and is_empty. Include docstrings
class Stack:
    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack.

        Args:
            item: The item to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack.

        Returns:
            The item at the top of the stack if the stack is not empty, otherwise None.
        """
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        """Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack if the stack is not empty, otherwise None.
        """
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        """Check if the stack is empty.
```

```python
class Stack:
    def is_empty(self):

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
# Test cases
if __name__ == '__main__':
    print("Testing Stack class:")
    stack = Stack()
    print(stack.is_empty())  # Should return True
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Should return 3
    print(stack.pop())  # Should return 3
    print(stack.peek())  # Should return 2
    print(stack.is_empty())  # Should return False
    stack.pop()
    stack.pop()
    print(stack.is_empty())  # Should return True
    print(stack.pop())  # Should return None (stack is empty)
    print(stack.peek())  # Should return None (stack is empty)
```

OUTPUT:

```
Testing Stack class:
True
3
3
2
False
True
None
None
```

EXPLANATION:

A stack is a **Last-In-First-Out (LIFO)** data structure where the most recently added element is removed first. It is useful for tasks like undo operations, expression evaluation, and backtracking. The push, pop, peek, and is_empty methods make it simple to manage items in order. Its simplicity makes it a fundamental building block in computer science.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

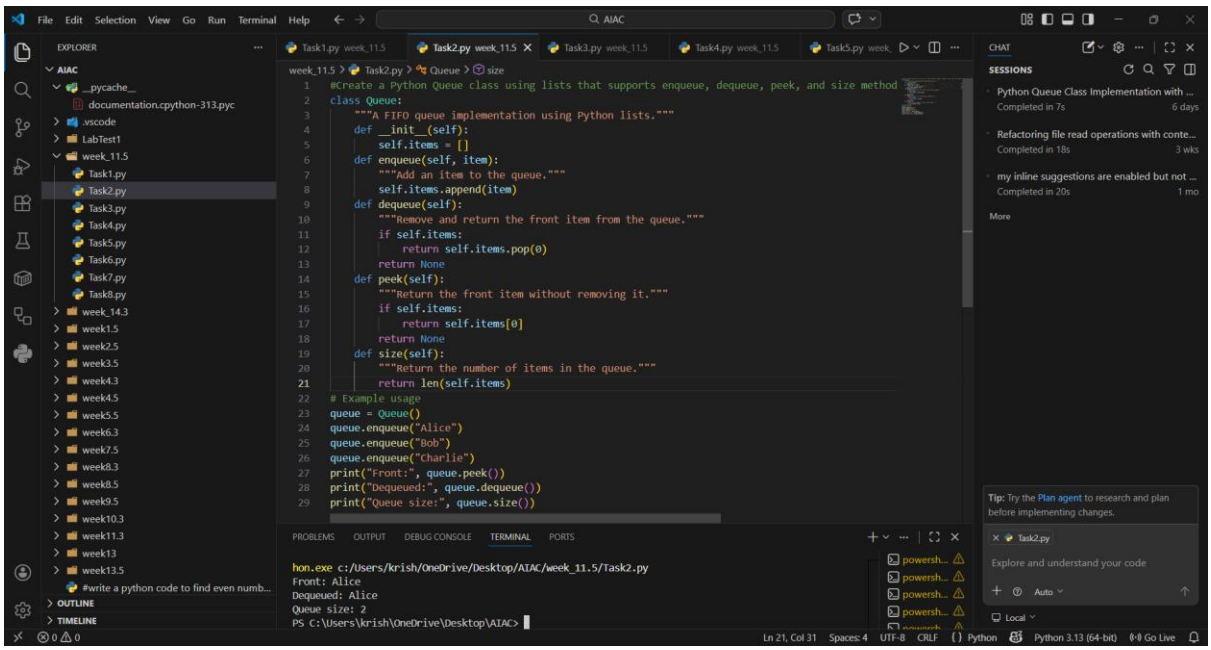
pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

PROMPT:" Create a Python Queue class using lists that supports enqueue, dequeue, peek, and size methods. Add docstrings for clarity"

CODE AND OUTPUT:



EXPLANATION:

A queue follows the **First-In-First-Out (FIFO)** principle, meaning the earliest added element is processed first. It is widely used in scheduling, buffering, and handling requests in order. The enqueue, dequeue, peek, and size methods ensure orderly processing. This structure models real-world scenarios like waiting lines.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

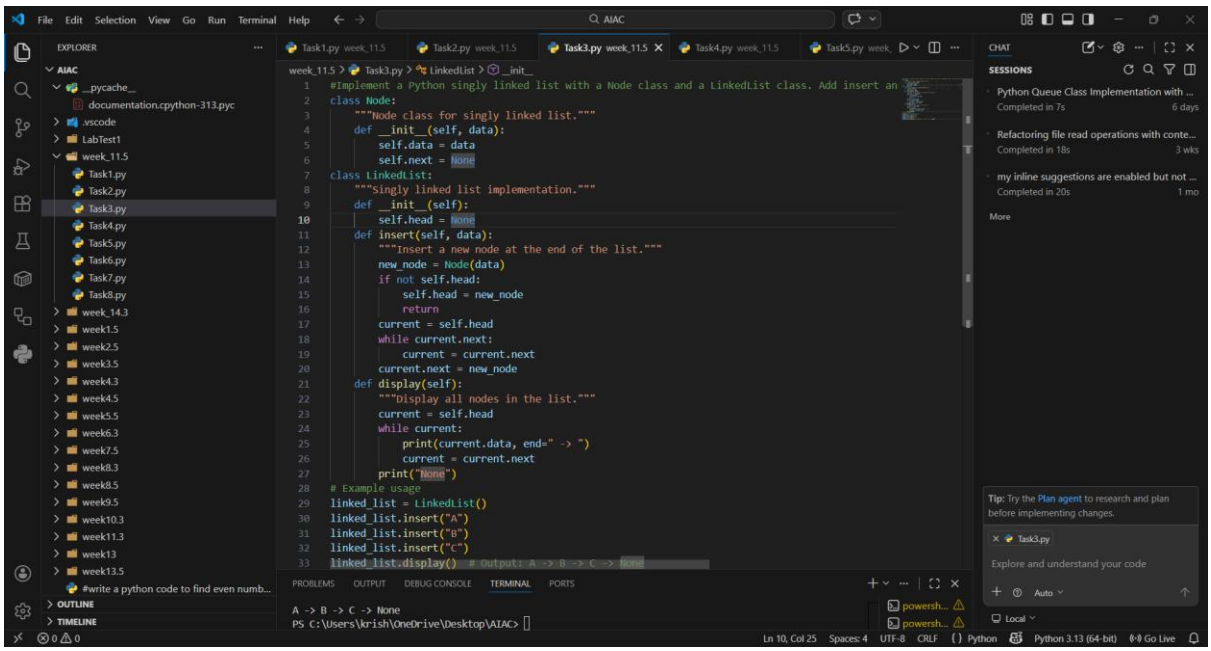
pass

Expected Output:

- **A working linked list implementation with clear method documentation.**

PROMPT:" Implement a Python singly linked list with a Node class and a LinkedList class. Add insert and display methods with proper documentation."

CODE AND OUTPUT:



EXPLANATION:

A linked list is a sequence of nodes where each node stores data and a reference to the next node. It allows dynamic memory allocation and efficient insertion at any position. The insert method adds new nodes, while display traverses the list. It is especially useful when the size of data is unpredictable.

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

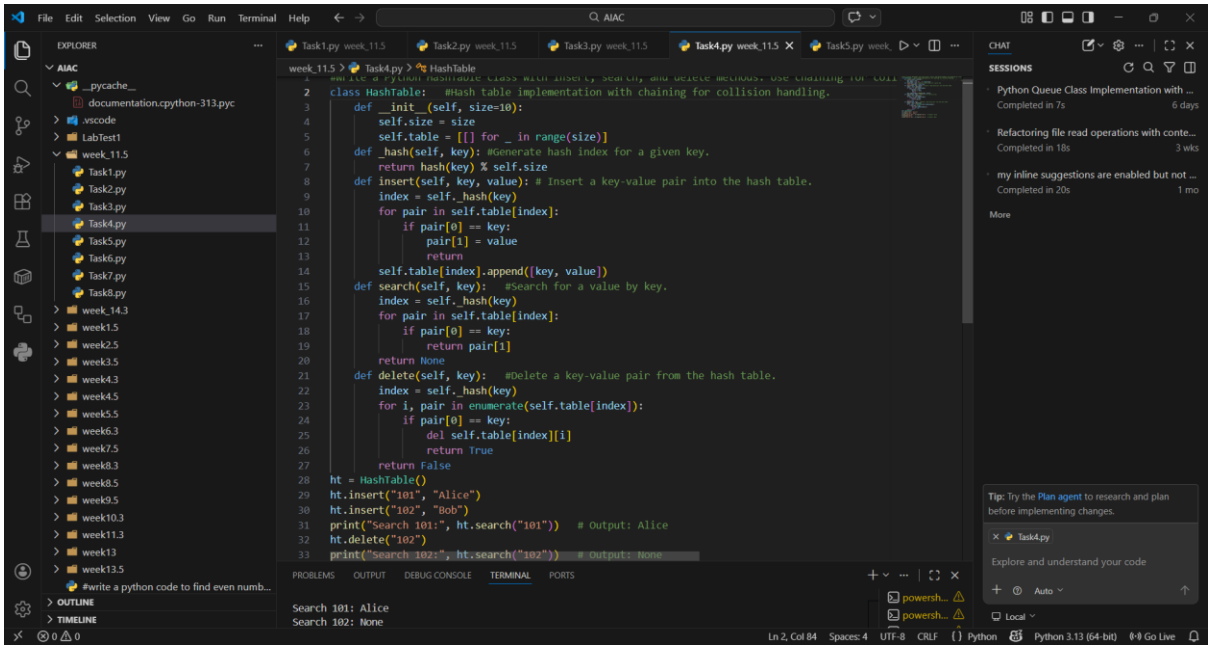
pass

Expected Output:

- **Collision handling using chaining, with well-commented methods.**

PROMPT:"Write a Python HashTable class with insert, search, and delete methods. Use chaining for collision handling and provide well-commented code."

CODE AND OUTPUT:



EXPLANATION:

A hash table stores key-value pairs and provides **fast lookup, insertion, and deletion** using hashing. Collisions are handled with chaining, ensuring multiple values can share the same index. It is ideal for storing records like IDs and names. Its efficiency makes it a backbone of databases and dictionaries.

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

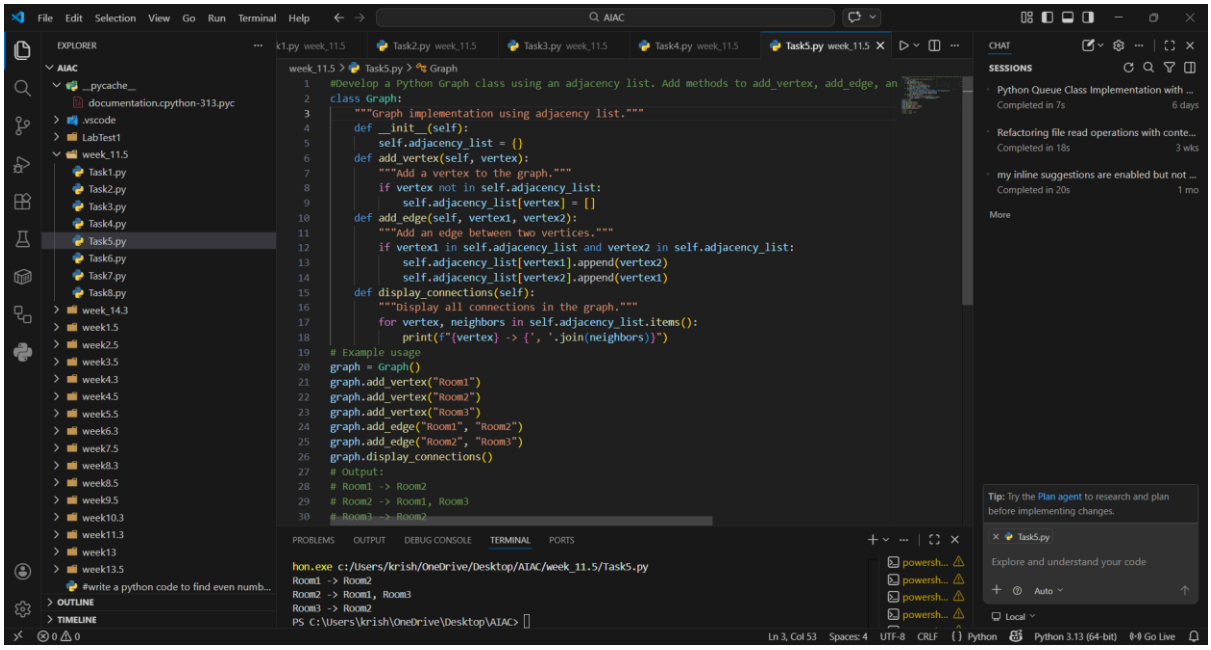
pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

PROMPT:"Develop a Python Graph class using an adjacency list. Add methods to add_vertex, add_edge, and display_connections. Include docstrings."

CODE AND OUTPUT:



EXPLANATION:

A graph represents relationships between entities using vertices and edges. The adjacency list implementation efficiently stores connections. It is useful in navigation systems, social networks, and network routing. Methods like add_vertex, add_edge, and display_connections make it easy to visualize connections.

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

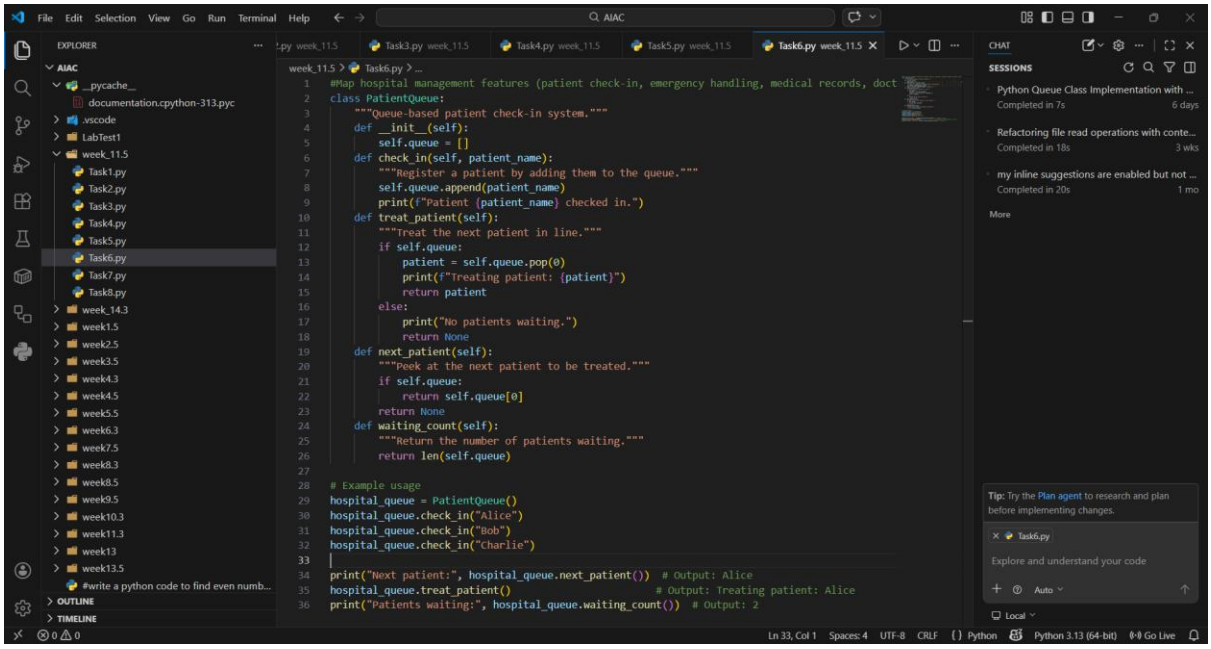
Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:"Map hospital management features (patient check-in, emergency handling, medical records, doctor appointment scheduling, hospital room navigation) to the most suitable data structures. Create a table with feature → data structure → justification. Then, implement one feature (e.g., Patient Check-In using a Queue) as a Python program with comments and docstrings."

CODE:



```python
#Map hospital management features (patient check-in, emergency handling, medical records, doct
class PatientQueue:
    """Queue-based patient check-in system."""
    def __init__(self):
        self.queue = []
    def check_in(self, patient_name):
        """Register a patient by adding them to the queue."""
        self.queue.append(patient_name)
        print(f"Patient {patient_name} checked in.")
    def treat_patient(self):
        """Treat the next patient in line."""
        if self.queue:
            patient = self.queue.pop(0)
            print(f"Treating patient: {patient}")
            return patient
        else:
            print("No patients waiting.")
            return None
    def next_patient(self):
        """Peek at the next patient to be treated."""
        if self.queue:
            return self.queue[0]
        return None
    def waiting_count(self):
        """Return the number of patients waiting."""
        return len(self.queue)

# Example usage
hospital_queue = PatientQueue()
hospital_queue.check_in("Alice")
hospital_queue.check_in("Bob")
hospital_queue.check_in("Charlie")

print("Next patient:", hospital_queue.next_patient())  # Output: Alice
hospital_queue.treat_patient()                         # Output: Treating patient: Alice
print("Patients waiting:", hospital_queue.waiting_count())  # Output: 2
```

OUTPUT:

```
Patient Alice checked in.
Patient Bob checked in.
Patient Charlie checked in.
Next patient: Alice
Treating patient: Alice
Patients waiting: 2
```

EXPLANATION:

The patient check-in system uses a queue to ensure patients are treated in order of arrival. This models real-world hospital workflows where fairness and order are essential. The check_in, treat_patient, and next_patient methods manage patient flow. It ensures smooth handling of routine cases.

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

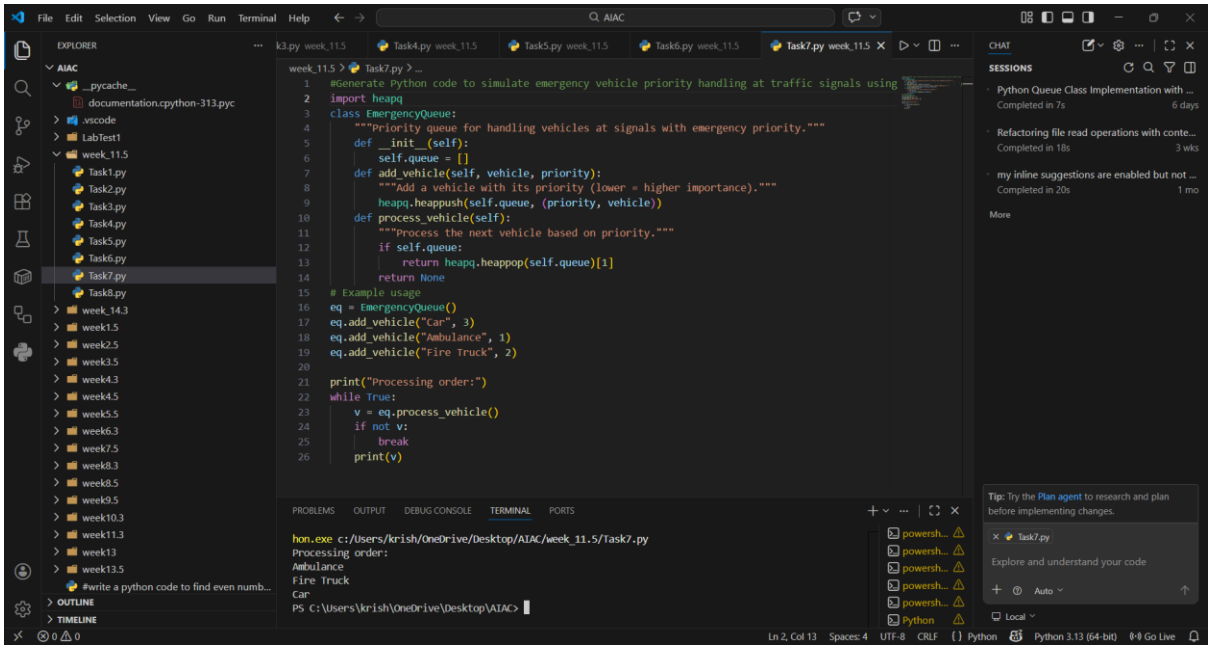
Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature**

with comments and docstrings.

PROMPT:" Generate Python code to simulate emergency vehicle priority handling at traffic signals using a priority queue."

CODE AND OUTPUT:



EXPLANATION:

This program uses a **priority queue** to manage vehicles at traffic signals. Emergency vehicles are given higher priority (lower number), ensuring they move first. The heapq module efficiently maintains the queue order. When processing, vehicles are dequeued based on priority, simulating real-world traffic control where ambulances and fire trucks are prioritized.

Task Description #8: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- **For each feature, select the most appropriate data structure from the list below:**
  - **o Stack**
  - **o Queue**
  - **o Priority Queue**
  - **o Linked List**
  - **o Binary Search Tree (BST)**
  - **o Graph**
  - **o Hash Table**
  - **o Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

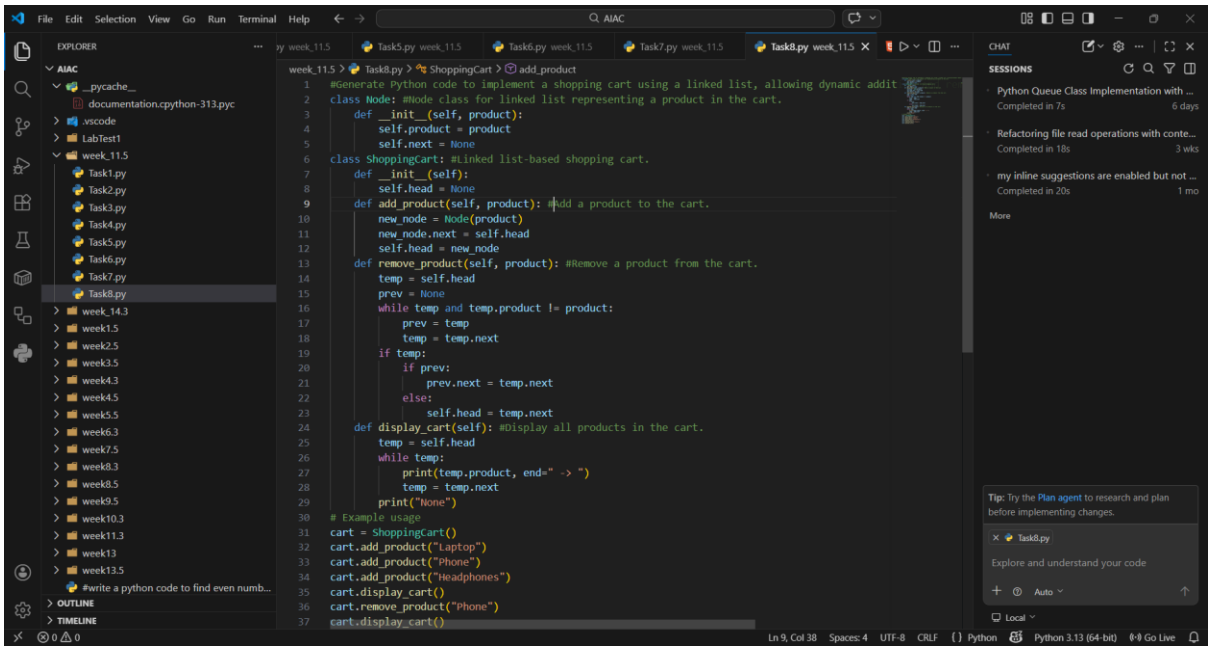
Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature**

with comments and docstrings.

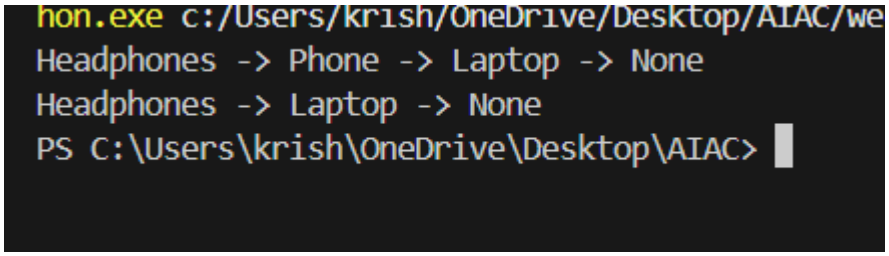
PROMPT:" Generate Python code to implement a shopping cart using a linked list, allowing dynamic addition and removal of products."

CODE:



```python
#Generate Python code to implement a shopping cart using a linked list, allowing dynamic addit
class Node: #Node class for linked list representing a product in the cart.
    def __init__(self, product):
        self.product = product
        self.next = None
class ShoppingCart: #Linked list-based shopping cart.
    def __init__(self):
        self.head = None
    def add_product(self, product): #Add a product to the cart.
        new_node = Node(product)
        new_node.next = self.head
        self.head = new_node
    def remove_product(self, product): #Remove a product from the cart.
        temp = self.head
        prev = None
        while temp and temp.product != product:
            prev = temp
            temp = temp.next
        if temp:
            if prev:
                prev.next = temp.next
            else:
                self.head = temp.next
    def display_cart(self): #Display all products in the cart.
        temp = self.head
        while temp:
            print(temp.product, end=" -> ")
            temp = temp.next
        print("None")
# Example usage
cart = ShoppingCart()
cart.add_product("Laptop")
cart.add_product("Phone")
cart.add_product("Headphones")
cart.display_cart()
cart.remove_product("Phone")
cart.display_cart()
```

OUTPUT:



```
hon.exe c:/Users/krish/OneDrive/Desktop/AIAC/we
Headphones -> Phone -> Laptop -> None
Headphones -> Laptop -> None
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

This program models a shopping cart using a **linked list**. Each product is stored as a node, allowing dynamic addition and removal. The cart can grow or shrink without fixed size limitations. The linked list structure ensures efficient updates, making it suitable for real-time e-commerce applications where customers frequently modify their cart.